Advanced Programming

Designing programs that scale with complexity

Mohammed Said BELAID
said.belaid@univ-usto.dz

30/09/2026



Outline

1 Advanced Programming?

2 Algorithms

3 What will we study?

4 Review: Basic data structures

5 Introduction to the next course



Definition

What makes a program "advanced"?

A program is not advanced simply because it uses an advanced programming language or contains a lot of code.



Definition

What makes a program "advanced"?

A program becomes advanced when we need to deal with complexity.

- Complex problem solving
- Larger programs
- More developers
- Performance requirements
- Larger amounts of data
- Code reuse
- Long term maintenance



Definition

Advanced Programming?

Advanced programming is the set of techniques for designing programs that remain efficient, maintainable, reusable, and understandable as their complexity grows.



Why Algorithms?

Algorithms are at the heart of computer science.

- Fundamental to many areas: networks, cryptography, databases, AI, ...
- Programming languages are tools, problem solving skills are the most important
- Algorithmic thinking helps us break down and solve complex problems
- Studying algorithms develops logical and analytical thinking



Prerequisites

This course assumes that you already have

- Experience with at least one programming language
- Basic programming constructs: conditions and loops
- Basic data structures: arrays, linked lists, ...
- Basic algorithmic concepts
- Basic understanding of complexity analysis



Chapters

Advanced Data Structures	Algorithmic Paradigms	Programming Paradigms	Best Programming Practices
How can we organize data efficiently?	How can we design efficient algorithms?	How can we structure our programs?	How can we make our code maintainable?



Advanced Data Structures

- Excludes elementary data structures (linked lists, trees, heap ...)
- Hash tables
- Dynamic arrays
- Tries
- Probabilistic data structures



Algorithmic Paradigms

How can we solve a problem?

- Brute force
- Divide and conquer
- Greedy algorithms
- Dynamic programming
- Backtracking
- Branch and bound
- Randomized algorithms



Programming Paradigms

- Procedural programming
- Object-oriented programming
- Functional programming
- ...



Best Programming Practices

Make software easier to understand, change, test, and maintain.

- Principles: SOLID, DRY, KISS, YAGNI
- Design: design patterns, antipatterns and code smells, refactoring
- Verification: unit testing, TDD



Basic data structures

- Arrays
- Linked lists
- Stacks
- Queues
- Trees
- Heaps (Tas)



Complexity Review

Data Structure	Search	Insert	Delete
Array (unsorted)	O(n)	O(1)[a]	O(n)
Sorted Array	O(log n)	O(n)	O(n)
Linked List	O(n)	O(1)[b]	O(1)[c]
Stack (push/pop)	–	O(1)	O(1)
Queue (enq/deq)	–	O(1)	O(1)
BST (average)	O(log n)	O(log n)	O(log n)
BST (worst)	O(n)	O(n)	O(n)
Heap	O(n)	O(log n)	O(log n)[d]

(a) append at the end, if capacity allows (b) at the head (c) given a reference to the node (d) removing the root (min/max)



Importance of Data Structures

Problem 1

An operating system manages many processes. Each process has a priority.

- Add a new process
- Find the highest-priority process
- Remove the selected process



Importance of Data Structures

Problem 1

An operating system manages many processes. Each process has a priority.

- Add a new process
- Find the highest-priority process
- Remove the selected process

Solution 1 (Array)

Find highest priority $\rightarrow O(n)$
Insert $\rightarrow O(1)$ (append at the end)
Remove $\rightarrow O(n)$



Importance of Data Structures

Problem 1

An operating system manages many processes. Each process has a priority.

- Add a new process
- Find the highest-priority process
- Remove the selected process

Solution 1 (Array)

Find highest priority $\rightarrow O(n)$
Insert $\rightarrow O(1)$ (append at the end)
Remove $\rightarrow O(n)$

Solution 2 (Sorted Array, ascending)

Find highest priority $\rightarrow O(1)$ (last element)
Insert $\rightarrow O(n)$
Remove $\rightarrow O(1)$ (remove the last element)



Importance of Data Structures

Problem 1

An operating system manages many processes. Each process has a priority.

- Add a new process
- Find the highest-priority process
- Remove the selected process

Solution 1 (Array)

Find highest priority $\rightarrow O(n)$
Insert $\rightarrow O(1)$ (append at the end)
Remove $\rightarrow O(n)$

Solution 2 (Sorted Array, ascending)

Find highest priority $\rightarrow O(1)$ (last element)
Insert $\rightarrow O(n)$
Remove $\rightarrow O(1)$ (remove the last element)

Solution 3 (Max Heap)

Find highest priority $\rightarrow O(1)$
Insert $\rightarrow O(log(n))$
Remove $\rightarrow O(log(n))$



Importance of Data Structures

Problem 2

We have a linked list containing messages waiting to be processed.
We frequently call the `size()` function.



Importance of Data Structures

Problem 2

We have a linked list containing messages waiting to be processed.
We frequently call the `size()` function.

Solution 1

```
int size() {
    Node current = head;
    int count = 0;
    while (current != null) {
        count++;
        current = current.next;
    }
    return count;
}
```

List size $\rightarrow O(n)$



Importance of Data Structures

Problem 2

We have a linked list containing messages waiting to be processed.
We frequently call the `size()` function.

Solution 2 (update the structure)

```
class LinkedList {
    Node head;
    int size;
    int size() {
        return size;
    }
}
```

Each insert `size++`. Each delete `size--`
List size $\rightarrow O(1)$

Solution 1

```
int size() {
    Node current = head;
    int count = 0;
    while (current != null) {
        count++;
        current = current.next;
    }
    return count;
}
```

List size $\rightarrow O(n)$



The Important Lesson

Data structure choice

The choice of a data structure depends on the operations we perform most often.

Before choosing a data structure

- What operations do we perform?
- How frequently do we perform them?
- How large can the data become?
- What are the performance requirements?



A simple problem

Find student

We have 1,000,000 students.
We want to find a student using their ID or name

The obvious solution

Search the list from beginning to end.
Worst case: $O(n)$



Can we do better?

Is there a more optimal approach?

What if we could go directly from:
Student ID or name to the student



The idea

Solution

We want a data structure that allows fast lookup using a key.
Key $\rightarrow$ Value
Student ID $\rightarrow$ Student



Next Course

Hash Tables implementation

